SOC Automation Home lab

Overview

Built a cloud-hosted, automated incident response environment designed to ingest endpoint telemetry, trigger high-fidelity alerts for credential dumping, orchestrate threat intelligence enrichment via VirusTotal, auto-generate tickets in a case management platform, and alert analysts instantly via email.

Technologies & Tools Utilized

- **OS/Infrastructure:** Windows 11 Enterprise (Endpoint), Ubuntu 24.04 LTS (Cloud Servers hosted on Vultr), VirtualBox (Hypervisor).
- **SIEM/EDR:** Wazuh (Log ingestion, active management, and alerting).
- **Endpoint Logging:** System Monitor (Sysmon) with Olaf Hartong's modular configuration.
- **SOAR Platform:** Shuffle.io (Automation orchestration workflow).
- **Threat Intelligence:** VirusTotal API (File hash analysis).
- **Case Management:** TheHive 5 (Incident ticketing and tracking).

Step-by-Step Implementation Breakdown

Phase 1: Endpoint Setup & Telemetry Configuration

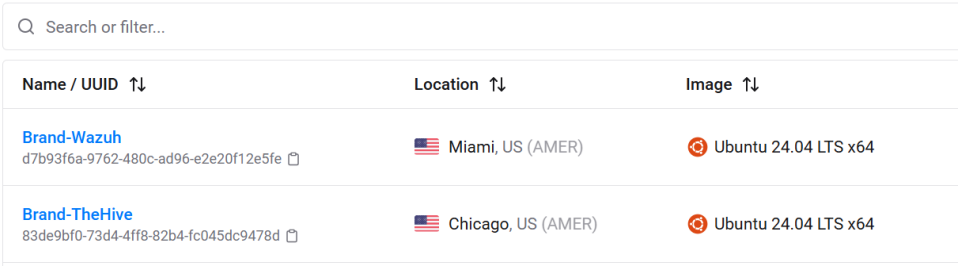
- **Virtual Machine Provisioning:** Setting up VirtualBox 7.1 with a Windows 11 Pro image.
- **Sysmon Deployment:** Installing Sysmon 64 and applying a comprehensive XML configuration (Olaf Hartong's template) to optimize the capturing of Event ID 1 (Process Creation) and file hashes.

Phase 2: Cloud Infrastructure & SIEM/Case Management Installation

- **Cloud Deployment:** Provisioned two instances on Vultr running Ubuntu 24.04 LTS for the central servers.
- **Wazuh Installation:** Utilizing the automated Wazuh installation assistant to deploy the indexer, server manager, and dashboard.
- **TheHive Deployment:** Installing prerequisites (Java Virtual Machine, Apache Cassandra, ElasticSearch) and mounting TheHive 5.5 package.
- **Network Security/Firewall Controls:** Configuring host-based firewalls via Uncomplicated Firewall (UFW) to allow authorized ingress on essential communication channels (Port 443

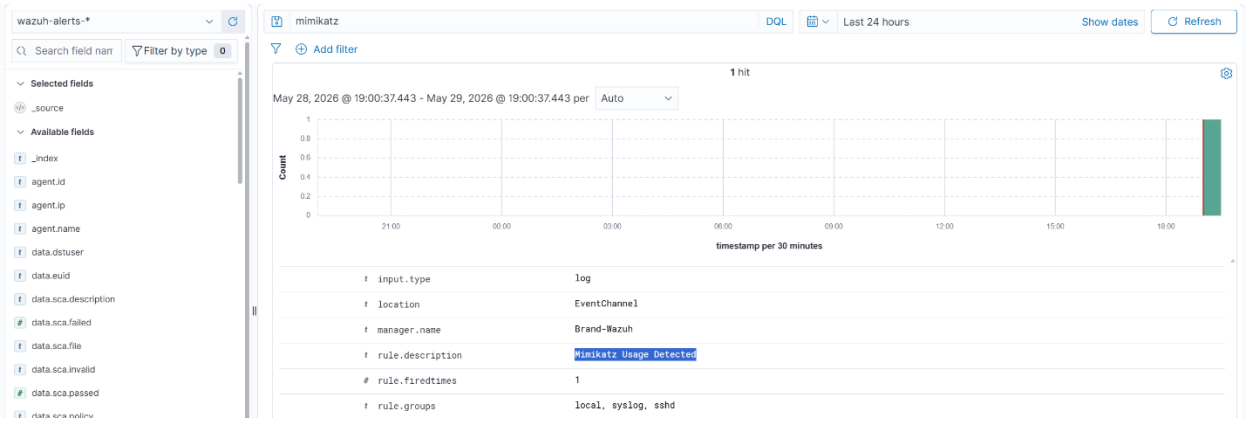
for HTTPS dashboard, Ports 1514/1515 for Wazuh Agent enrollment, and Port 9000 for TheHive).

Instances



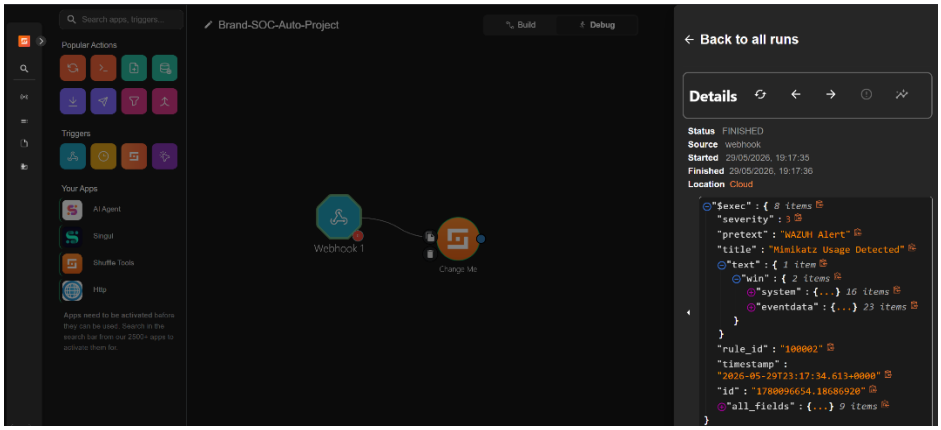
Phase 3: Agent Enrollment & Custom Detection Engineering

- **Wazuh Agent Deployment:** Installing the MSI agent on the Windows 11 endpoint using PowerShell, configuring ossec.conf on the agent to target the specific cloud manager IP, and verifying the active communication path.
- **Sysmon Log Forwarding:** Editing the local endpoint ossec.conf to target and pull specifically from the Sysmon operational event channel name.
- **Custom Ruleset Authoring:** Creating a customized detection rule inside the Wazuh manager's local_rules.xml. Outline the logic used: matching Sysmon Event ID 1 (Process Creation) and looking for an original_file_name string match of mimikats.exe.
- **Telemetry Testing:** Simulating an adversary profile by executing Mimikats binaries on the endpoint (bypassing Windows Defender exclusions) to successfully validate that the SIEM captures and tags the custom rule alert ID (e.g., 100002).

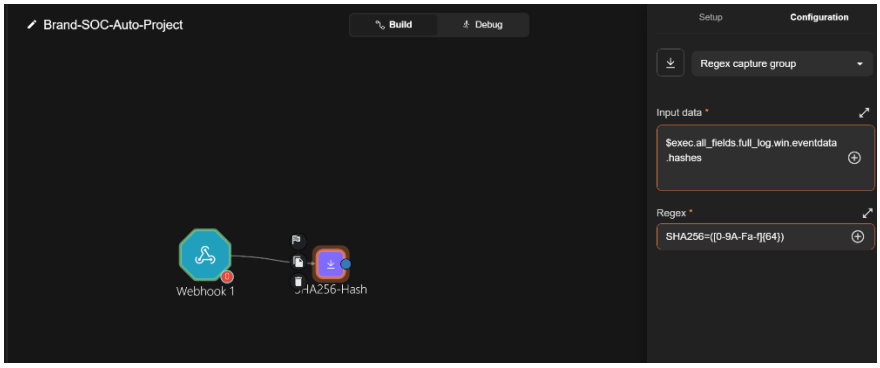


Phase 4: SOAR Orchestration & Playbook Automation

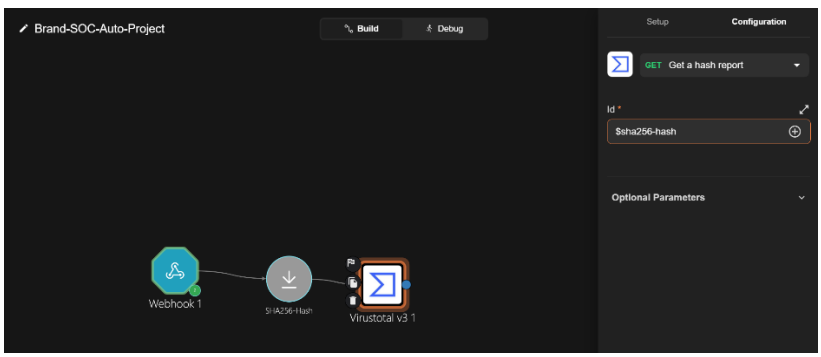
- **Wazuh-to-SOAR Integration:** Configuring the global <integration> block in the Wazuh Manager's ossec.conf to forward JSON-formatted alerts matching your custom rule ID straight to a Shuffle webhook URI.
- **Workflow Logic Construction:**
    - *Step 1 (Ingestion):* Catching the inbound Webhook stream.



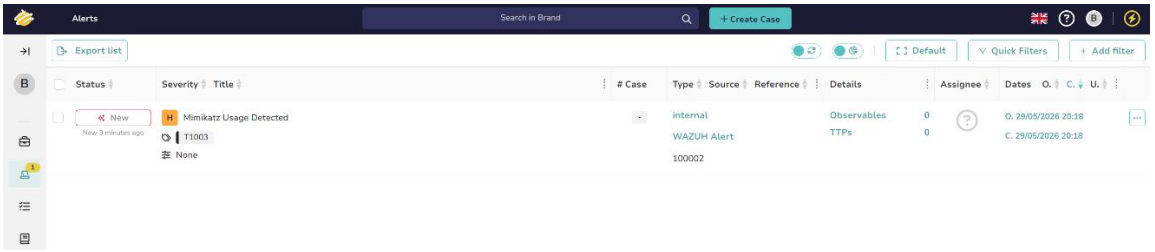
    - *Step 2 (Regex Extraction):* Using regex execution arrays within Shuffle Tools to strip out the exact SHA256 string from the complex logs.



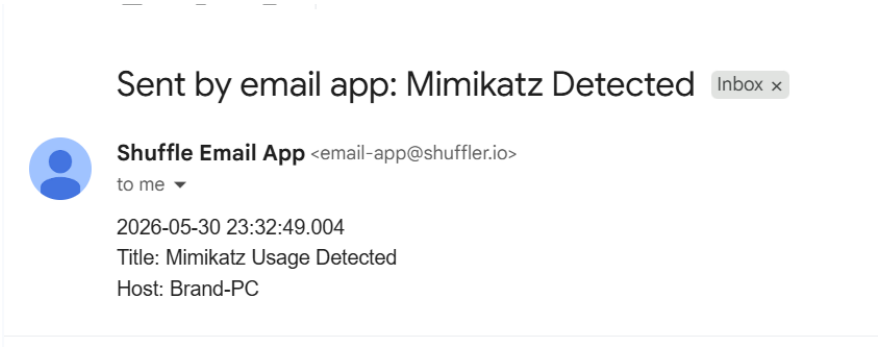
    - *Step 3 (Threat Intel Enrichment):* Issuing automated API requests (GET /hash_report) to VirusTotal using the clean extracted hash.



- *Step 4 (Ticketing Deployment):* Programmatically sending an advanced JSON payload to TheHive API via a dedicated Service Account API key to dynamically establish a new incident alert block complete with host details and severity metrics.



- *Step 5 (Analyst Alerting):* Constructing an email notification action to blast critical data directly to the security team's inbox upon execution.



Troubleshooting

A problem that I ran into was getting a lot of false positives as alerts and therefore getting emails sent that shouldn't have been, it was getting alerts for every windows event that was happening on the machine. I figured out it was a problem with my ossec.conf and local_rules.xml files on my Wazuh manager. Below are the changes that I made to stop getting false positives.

```
<integration>
  <name>shuffle</name>
  <hook_url>https://shuffler.io/api/v1/hooks/webhook_f4f54021-547a-4bb8-9c6c-b887666</hook_url>
  <rule_id>100002,100003,100004</rule_id>
  <alert_format>json</alert_format>
</integration>
```

```
<rule id="100002" level="15">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.originalFileName" type="pcre2">(?i)mimikatz</field>
  <description>Mimikatz Usage Detected</description>
  <mitre>
      <id>T1003</id>
  </mitre>
</rule>

<rule id="100003" level="15">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.commandLine" type="pcre2">(?i)(sekurlsa|logonpasswords|>
  <description>Mimikatz Usage Detected - known command arguments</description>
  <mitre>
    <id>T1003</id>
  </mitre>
</rule>

<!-- LSASS memory access (the real credential dump event) -->
<rule id="100004" level="15">
  <if_group>sysmon_event10</if_group>
  <field name="win.eventdata.targetImage" type="pcre2">(?i)lsass\.exe</field>
  <field name="win.eventdata.grantedAccess" type="pcre2">0x1010|0x1410|0x147a|0x1fff>
  <description>Mimikatz Usage Detected - LSASS memory access</description>
  <mitre>
    <id>T1003</id>
  </mitre>
</rule>
```

Before these changes I only had rule 100002 which was matching normal windows activity as well as Mimikatz.